

Работа с innerHTML, outerHTML

В прошлом уроке мы с вами учились считывать и записывать значения атрибутов, а теперь будем делать тоже самое, но уже со **внутренним содержимым** тегов. Это делается с помощью свойства **innerHTML**, которое можно считывать и перезаписывать, меняя тем самым содержимое тегов.

В примере ниже у нас есть абзац. Давайте сменим ему текст с помощью свойства **innerHTML**. Для этого получим ссылку на этот абзац с помощью **getElementById** в переменную **elem**. Затем таким образом - **elem.innerHTML** = 'Новый текст' - запишем в него новый текст:

```
<p id="test">Текст абзаца.</p>
<input type="submit" onclick="func()">
//Функция func срабатывает по клику на кнопку input:
function func() {
    var elem = document.getElementById('test'); //получаем наш абзац
    elem.innerHTML = '!'; //записываем в него новый текст
}
```

HTML код станет выглядеть так:

```
<p id="test">!</p>
<input type="submit" onclick="func()">
```

Можно записывать не только текст, но и **теги** и они будут работать (в нашем случае текст станет жирным):

```
<p id="test">Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('test');
    elem.innerHTML = '<b>!</b>';
}
```

HTML код станет выглядеть так:

```
<p id="test"><b>!</b></p>
<input type="submit" onclick="func()">
```

Кроме свойства **innerHTML** существует также свойство **outerHTML**, которое перезаписывает не только внутренний текст тега, но и **сам тег**. Смотрите пример:

```
<p id="test">Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('test');
    elem.outerHTML = '<b>!</b>';
}
```

HTML код станет выглядеть так:

```
<b>!</b>
```

```
<input type="submit" onclick="func()">
```

Как вы видите, наш **абзац исчез** и заменился на тег **b**.

Работа с `getElementsByName`

Во всех примерах выше мы с вами работали с методом `getElementById`, который получал только одно свойство по его атрибуту `id`. Однако существуют и другие способы получения свойств, к примеру, с помощью метода `getElementsByName`, который получает группу тегов по их имени. Например, можно получить все абзацы **p** или все заголовки **h2**.

Учтите только один нюанс - наш метод возвращает не одно свойство, а **массив** свойств. Получим, к примеру, все абзацы в переменную **elems**:

```
<p>Текст абзаца.</p>
<p>Текст абзаца.</p>
<p>Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
    var elems = document.getElementsByName('p');
}
```

В переменной **elems** теперь лежит массив свойств, это значит, что **elems[0]** - это первый абзац, **elems[1]** - второй и так далее (нумерация с нуля).

Мы можем убедиться в этом, если сделаем так: `elems[0].innerHTML = '!'` - в этом случае **первый** абзац сменит свой текст.

Давайте поменяем текст всех трех абзацев:

```
<p>Текст абзаца.</p>
<p>Текст абзаца.</p>
<p>Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
    //Получаем массив абзацев в переменную elems:
    var elems = document.getElementsByName('p');

    elems[0].innerHTML = '1'; //первому абзацу ставим текст '1'
    elems[1].innerHTML = '2'; //второму абзацу ставим текст '2'
    elems[2].innerHTML = '3'; //третьему абзацу ставим текст '3'
}
```

HTML код станет выглядеть так:

```
<p>1</p>
<p>2</p>
<p>3</p>
```

```
<input type="submit" onclick="func()">
```

Учтите, что менять одновременно все свойства как-нибудь так `elems.innerHTML = '!'` не получится (потому что **elems** - массив). Необходимо или вручную перебрать все свойства, как это сделано в примере выше, или воспользоваться **циклом**.

В следующем примере мы сменим содержимое всех абзацев на страницы на '!', воспользовавшись для этого циклом **for**:

```
<p>Текст абзаца.</p>
```

```
<p>Текст абзаца.</p>
```

```
<p>Текст абзаца.</p>
```

```
<input type="submit" onclick="func()">
```

```
function func() {  
    var elems = document.getElementsByTagName('p');  
    for (var i = 0; i < elems.length; i++) {  
        elems[i].innerHTML = '!';  
    }  
}
```

HTML код станет выглядеть так:

```
<p>!</p>
```

```
<p>!</p>
```

```
<p>!</p>
```

```
<input type="submit" onclick="func()">
```

Обращение к свойствам через свойства **document**

Некоторые теги страницы можно не получать методами типа **getElementById**, а обратиться к ним как к свойствам объекта **document**. К примеру, к тегу **<body>** можно обратиться так: **document.body**, а к тегу **<head>** - вот так: **document.head**. Такое работает не для всех тегов, а только для избранных.

Учтите, что нельзя получить доступ к тому свойству, которого еще не существует в момент выполнения скрипта. Поэтому, если скрипт находится в **<head>**, то в нем недоступен **document.body**.

В следующем примере первый alert выведет **null**:

```
<!DOCTYPE HTML>
```

```
<html>
```

```
    <head>
```

```
        <script>
```

```
            alert(document.body); //null, так как body еще нет
```

```
        </script>
```

```
    </head>
```

```
    <body>
```

```

        <script>
            alert(document.body); //body уже есть
        </script>
    </body>
</html>

```

Работа с формами

Можно получить все **HTML формы** на странице через **document.forms**. В результате мы получим массив форм, будто он был получен с помощью **getElementsByTagName** (точнее это будет не массив, а **псевдомассив (или коллекция)** свойств).

Мы можем, к примеру, обратиться к любой форме, как к энному свойству массива, например, к форме с номером 0:

```

<body>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
</body>
document.forms[0].innerHTML = '!';

```

HTML код станет выглядеть так:

```

<body>
    <form>
        !
    </form>
    <form>
        <input type="text" value="1">
        <input type="text" value="2">
        <input type="text" value="2">
    </form>
</body>

```

А можем перебрать все формы с помощью цикла:

```

<body>

```

```

    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
</body>
var forms = document.forms;

for (var i = 0; i < forms.length; i++) {
    forms[i].innerHTML = '!';
}

```

HTML код станет выглядеть так:

```

<body>
    <form>
        !
    </form>
    <form>
        !
    </form>
</body>

```

Следующим образом - **document.forms[i].elements** - можно найти общее количество форм на странице.

Кроме того, к формам можно обращаться по имени вместо номера. Это имя задается в атрибуте **name** тега **<form>**. Например, если форме дать имя **test**, то к ней можно обратиться так - **document.forms['test']** или так - **document.forms.test**.

В следующем примере форме с именем **form1** поставим текст '!', а форме с именем **form2** - текст '?' (двумя разными способами):

```

<body>
    <form name="form1">
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form name="form2">

```

```
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
</body>
document.forms.form1.innerHTML = '!';
document.forms['form2'].innerHTML = '?';
```

HTML код станет выглядеть так:

```
<body>
    <form name="form1">
        !
    </form>
    <form name="form2">
        ?
    </form>
</body>
```

Обращение к свойствам форм

У каждой формы есть свойство **elements**, которое содержит массив свойств формы. С его помощью можно обратиться к определенному свойству формы по его номеру.

Для примера давайте обратимся к форме номер 1, а затем к ее свойству номер 0:

```
<body>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
</body>
document.forms[1].elements[0].value = '!';
```

HTML код станет выглядеть так:

```
<body>
    <form>
        <input value="1">
```

```

        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="!">
        <input value="2">
        <input value="3">
    </form>
</body>

```

Можно также перебрать все свойства формы с помощью цикла. Давайте получим форму с номером 1 и всем свойствам этой формы поставим **value** '!':

```

<body>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
</body>

var elements = document.forms[1].elements;

for (var i = 0; i < elements.length; i++) {
    elements[i] = '!';
}

```

HTML код станет выглядеть так:

```

<body>
    <form>
        <input value="1">
        <input value="2">
        <input value="3">
    </form>
    <form>
        <input value="!">
        <input value="!">

```

```
        <input value="!">
    </form>
</body>
```

При необходимости с помощью двух вложенных циклов можно перебрать все формы и все свойства внутри них.

К свойствам формы, так же, как и к самим формам, можно обращаться по имени из атрибута **name**:

```
<body>
    <form name="form1">
        <input value="1" name="input1">
        <input value="2" name="input2">
        <input value="3" name="input3">
    </form>
</body>
```

```
document.forms.form1.elements.input1.value = '!';
```

HTML код станет выглядеть так:

```
<body>
    <form name="form1">
        <input value="!" name="input1">
        <input value="2" name="input2">
        <input value="3" name="input3">
    </form>
</body>
```

Это не все приемы работы с формами, при необходимости смотрите подробности в справочнике JavaScript тут: [работа с формами](#).

Продвинутая работа с атрибутами

Вы уже знаете один из способов работы с атрибутами тегов, однако есть **альтернативный** вариант - с помощью методов **getAttribute**, **setAttribute**, **removeAttribute** и **hasAttribute**.

Метод **getAttribute** считывает значение указанного атрибута (и атрибут **class** тут уже не будет исключением), **setAttribute** - записывает новое значение в атрибут, **removeAttribute** удаляет атрибут совсем, а **hasAttribute** проверяет наличие или отсутствие атрибута у свойства.

Давайте получим значение атрибута **value** для инпута с помощью **getAttribute**:

```
<input type="text" value="!" id="test">
<input type="submit" onclick="func()">

function func() {
    var elem = document.getElementById('test');
    alert(elem.getAttribute('value'));
```



```
}
```

А теперь запишем новое значение 'www' в атрибут **value**:

```
<input type="text" value="!" id="test">
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('test');
    elem.setAttribute('value', 'www');
}
```

Удалим атрибут **value** совсем:

```
<input type="text" value="!" id="test">
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('test');
    elem.removeAttribute('value');
}
```

HTML код станет выглядеть так (исчезнет атрибут value):

```
<input type="text" id="test">
<input type="submit" onclick="func()">
```

Ну, а теперь проверим наличие атрибута **value** у нашего свойства:

```
<input type="text" value="!" id="test">
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('test');
    alert(elem.hasAttribute('value')); //выведет true
}
```

Получение свойств по их классу

Вы уже знаете метод **getElementsByTagName**, который получает **группу** свойств по названию тега. Существует **аналогичный** метод **getElementsByClassName**, который получает свойства по их **классу** (то есть по содержимому **атрибута class**).

Так же, как и для других подобных методов, **нельзя** обратиться ко всем свойствам **одновременно**, нужно использовать **цикл**. В примере ниже мы получаем все свойства с классом **www** и ставим им текст '!':

```
<p class="www">Текст абзаца.</p>
<p class="www">Текст абзаца.</p>
<p class="www">Текст абзаца.</p>
<p>Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
```

```

        var elems = document.getElementsByClassName('www');
        for (var i = 0; i < elems.length; i++) {
            elems[i].innerHTML = '!';
        }
    }
}

```

HTML код станет выглядеть так:

```

<p class="www">!</p>
<p class="www">!</p>
<p class="www">!</p>
<p>Текст абзаца.</p>
<input type="submit" onclick="func()">

```

Получение свойств селектором CSS

В настоящее время в JavaScript появились более **продвинутые** методы получения свойств, в которых можно использовать любые **селекторы CSS**.

Для этого можно использовать два метода: **querySelector** и **querySelectorAll**. Метод **querySelector** является аналогом **getElementById**, только принимает не **id** свойства, а любой **селектор**, а возвращает ссылку на **один** найденный свойство (если под селектор попадает много свойств, то она вернет ссылку на первый свойство).

В примере ниже с помощью **querySelector** мы получаем инпут с классом **test** и выводим на экран содержимое его атрибута **value**:

```

<input type="text" value="!" class="test">
<input type="submit" onclick="func()">
function func() {
    var elem = document.querySelector('input.test');
    alert(elem.getAttribute('value'));
}

```

А сейчас мы получим свойство с **id** равным **test**:

```

<input type="text" value="!" id="test">
<input type="submit" onclick="func()">
function func() {
    var elem = document.querySelector('#test');
    alert(elem.getAttribute('value'));
}

```

Давайте теперь разберем **querySelectorAll**. Этот метод, в отличие от **querySelector**, получает **группу** свойств и работать с ними нужно как с группой (перебирать **циклом** и т.п.). В примере ниже мы получаем все свойства с классом **www** и ставим им текст '!':

```

<p class="www">Текст абзаца.</p>
<p class="www">Текст абзаца.</p>
<p class="www">Текст абзаца.</p>

```

```

<p>Текст абзаца.</p>
<input type="submit" onclick="func()">
function func() {
    var elems = document.querySelectorAll('.www');
    for (var i = 0; i < elems.length; i++) {
        elems[i].innerHTML = '!';
    }
}

```

HTML код станет выглядеть так:

```

<p class="www">!</p>
<p class="www">!</p>
<p class="www">!</p>
<p>Текст абзаца.</p>
<input type="submit" onclick="func()">

```

Атрибуты data-

В языке HTML5 разрешено добавлять свои атрибуты тегам, при этом они должны начинаться с **data-**, а затем должно идти любое название атрибута, которое вам удобно (это будет работать корректно во всех браузерах, кроме IE10 и ниже).

Если вы хотите обратиться к таким атрибутам как с свойствам объекта, то это делается не на прямую, а при помощи специального свойства **dataset**:

```

<div id="elem" data-price="1000" data-product-number="5">
    Товар Джинсы
</div>
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('elem');

    alert(elem.dataset.price); //выведет 1000
    alert(elem.dataset.productNumber); //выведет 5
}

```

Обратите внимание – атрибут **data-price** превратился в **dataset.price**, а **data-product-number** превратился в **productNumber**.

К таким атрибутам можно также обращаться с помощью методов типа **getAttribute**, в этом случае следует писать полное название атрибута:

```

<div id="elem" data-price="1000" data-product-number="5">
    Товар Джинсы
</div>
<input type="submit" onclick="func()">

```

```
function func() {
    var elem = document.getElementById('elem');

    alert(elem.getAttribute('data-price')); //выведет 1000
    alert(elem.getAttribute('data-product-number')); //выведет 5
}
```

Разница между способами получения атрибутов

На самом деле работа с атрибутами через свойства (так: `elem.value`) и через `getAttribute` (так: `elem.getAttribute('value')`) - это не одно и то же.

При изменении свойства `elem.value` атрибут `elem.getAttribute('value')` не меняется:

```
<input id="test" type="text" value="old">
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('elem');

    elem.value = 'new'; //поменяли свойство
    alert(elem.getAttribute('value')); //выведет 'old' - не изменилось!
}
```

То есть, изменение свойства на атрибут не влияет, он остается таким же.

А вот изменение атрибута обновляет свойство:

```
<input id="test" type="text" value="old">
<input type="submit" onclick="func()">
function func() {
    var elem = document.getElementById('elem');

    elem.setAttribute('value', 'new'); //поменяли атрибут
    alert(elem.value); //выведет 'new' - elem.value изменилось!
}
```

Эту особенность можно использовать.

Получается, что атрибут `elem.getAttribute('value')` хранит исходное значение даже после того, как пользователь заполнил поле и свойство изменилось.

Например, можно взять изначальное значение из атрибута и сравнить со свойством, чтобы узнать, изменилось ли значение. А при необходимости и перезаписать свойство атрибутом, отменив изменения.

**Примеры решения задач :Весь код ниже вставляйте
и проверяйте что будет происходить**

Задача . Изменение текста в span

Задача. Повторите поведение кнопки по нажатию на нее (она меняет текст в span):

Нажми на меня !!!

Решение:

```
<button onclick="buttonClick()">Нажми на меня</button>
<span id="elem">Это span с текстом.</span>
function buttonClick() {
    var elem = document.getElementById('elem');
    elem.innerHTML = '!!!';
}
```

Задача . Замена span на тег b без изменения текста

Задача. Повторите поведение кнопки по нажатию на нее (она меняет span на тег b, не изменяя при этом текст внутри тега):

Нажми на меня **Это span с текстом.**

Решение: просто так заменить теги, не трогая их внутреннего содержимого, невозможно. В данном случае необходимо найти внутреннее содержимое тега span, а затем заменить этот span на тег b, внутри которого будет стоять то, что было внутри span. Чтобы тег span исчез совсем, нужно поменять его outerHTML, а не innerHTML (в этом случае тег b оказался бы внутри span, а это нам не нужно по условию задачи):

```
<button onclick="buttonClick()">Нажми на меня</button>
<span id="elem">Это span с текстом.</span>
function buttonClick() {
    var elem = document.getElementById('elem');
    elem.outerHTML = '<b>' + elem.innerHTML + '</b>';
}
```

Задача . На getElementsByTagName

Задача. Дан HTML код (см. под задачей). Поменяйте содержимое **абзацев** на их порядковый номер в коде.

```
<h2>Заголовок, не поменяется.</h2>
<p>Абзац, поменяется.</p>
<p>Абзац, поменяется.</p>
<p>Абзац, поменяется.</p>
```

Решение: создадим кнопку, по нажатию на которую будет вызываться функция func, которая выполнит указанные в условии задачи действия:

```
<input type="submit" onclick="func()">
```

Получим теперь элементы по имени тега и переберем их циклом, используя метод getElementsByTagName:

```
function func() {
    var elems = document.getElementsByTagName('p');
    for (var i = 0; i < elems.length; i++) {
        elems[i].innerHTML = i+1;
    }
}
```

HTML код станет выглядеть так:

```
<h2>Заголовок, не поменяется.</h2>
<p>1</p>
<p>2</p>
<p>3</p>
<input type="submit" onclick="func()">
```

Задача . На getElementsByClassName

Задача. Дан HTML код (см. под задачей). Поменяйте содержимое элементов с классом **zzz** на их порядковый номер в коде.

```
<h2 class="zzz">Заголовок с классом zzz.</h2>
<p class="zzz">Абзац с классом zzz.</p>
<p class="zzz">Абзац с классом zzz.</p>
<p>Просто абзац, не поменяется.</p>
```

Решение: создадим кнопку, по нажатию на которую будет вызываться функция func, которая выполнит указанные в условии задачи действия:

```
<input type="submit" onclick="func()">
```

Получим теперь элементы по имени класса и переберем их циклом, используя метод `getElementsByClassName`:

```
function func() {
    var elems = document.getElementsByClassName('zzz');
    for (var i = 0; i < elems.length; i++) {
        elems[i].innerHTML = i+1;
    }
}
```

HTML код станет выглядеть так:

```
<h2 class="zzz">1</h2>
<p class="zzz">2</p>
<p class="zzz">3</p>
<p class="zzz">4</p>
<p>Просто абзац, не поменяется.</p>
<input type="submit" onclick="func()">
```

Задача . На querySelectorAll

Задача. Дан HTML код (см. под задачей). Поменяйте содержимое абзацев с классом **zzz** на их порядковый номер в коде.

```
<h2 class="zzz">Заголовок с классом zzz, не поменяется.</h2>
<p class="zzz">Абзац с классом zzz.</p>
<p class="zzz">Абзац с классом zzz.</p>
<p>Просто абзац, не поменяется.</p>
```

Решение: создадим кнопку, по нажатию на которую будет вызываться функция func, которая выполнит указанные в условии задачи действия:

```
<input type="submit" onclick="func()">
```

Получим теперь элементы с помощью селектора **p.zzz** (абзацы с классом **zzz**) и переберем их циклом, используя метод **querySelectorAll**:

```
function func() {
    var elems = document.querySelectorAll('p.zzz');
    for (var i = 0; i < elems.length; i++) {
        elems[i].innerHTML = i+1;
    }
}
```

HTML код станет выглядеть так:

```
<h2 class="zzz">Заголовок с классом zzz, не поменяется.</h2>
<p class="zzz">1</p>
<p class="zzz">2</p>
<p class="zzz">3</p>
<p>Просто абзац, не поменяется.</p>
<input type="submit" onclick="func()">
```

Задачи на события:

Свойства innerHTML, outerHTML

Для решения задач данного блока вам понадобятся следующие свойства: innerHTML, outerHTML.

Повторите страницу по данному образцу: При клике на кнопку у вас должен измениться текст и стать жирным

Ку-ку! А я жирный!

Нажмите на меня!

Повторите страницу по данному по образцу: При нажатии на кнопку текст поменяется и станет заголовком h3

При нажатии на кнопку текст в абзаце поменяется.

Нажмите на меня!

Повторите страницу по данному по образцу: При нажатии на кнопку текст останется , но станет заголовком h3

При нажатии на кнопку абзац станет h3, но текст останется.

Нажмите на меня!

Метод `getElementsByName`

Для решения задач данного блока вам понадобятся следующие методы: `getElementsByName`.

Повторите страницу по данному по образцу: весь текст изменится кроме заголовка h2

При нажатии на кнопку текст в каждом абзаце поменяется.

При нажатии на кнопку текст в каждом абзаце поменяется.

При нажатии на кнопку текст в каждом абзаце поменяется.

При нажатии на кнопку текст в каждом абзаце поменяется.

А я h2 и не поменяюсь!

Нажмите на меня!

Метод `getElementsByClassName`

Для решения задач данного блока вам понадобятся следующие методы: `getElementsByClassName`.

Дан HTML код (см. под задачей). Поменяйте содержимое элементов с классом **www** на их порядковый номер в коде.

```
<h2 class="www">Заголовок с классом www.</h2>
<p class="www">Абзац с классом www.</p>
<p class="www">Абзац с классом www.</p>
<p>Просто абзац, не поменяется.</p>
```

Метод `querySelectorAll`

Для решения задач данного блока вам понадобятся следующие методы: `querySelectorAll`.

Дан HTML код (см. под задачей). Поменяйте содержимое абзацев с классом **www** на их порядковый номер в коде.

```
<h2 class="www">Заголовок с классом www, не поменяется.</h2>
<p class="www">Абзац с классом www.</p>
<p class="www">Абзац с классом www.</p>
<p class="www">Абзац с классом www.</p>
<p>Просто абзац, не поменяется.</p>
```

Задачи

Повторите страницу по данному образцу: При вводе текста в инпут он должен выводиться справа от инпута

какой-то текст | какой-то текст

Закрепление пройденного

Повторите страницу по данному образцу: При клике на кнопку он блокируется и меняется текст

При нажатии на кнопку текст в абзаце поменяется.

Нажмите на меня!

То есть будет так:

Привет, мир!

О, на меня нельзя больше нажать!

Повторите страницу по данному образцу (при нажатии на кнопку текст в каждом абзаце поменяется на порядковый номер абзаца):

Повторите страницу по данному образцу (при нажатии на кнопку текст в каждом абзаце поменяется - спереди добавится порядковый номер абзаца):

Работа с Датами джаваскрипт

Работа с **датами** в JavaScript ведется в основном через объект **Date**. Этот объект создается следующим образом:

```
var date = new Date();
```

Теперь переменная **date** - это **объект** с датой, который хранит в себе *текущий момент времени* (секунду, минуту, час и так далее). С помощью специальных функций мы можем получить нужные нам характеристики времени, например, текущий час, текущий день или текущий месяц.

К примеру, текущий час можно получить так: `date.getHours()`, а текущий месяц - вот так `date.getMonth()`. Смотрите все варианты:

```
var date = new Date();
```

```
alert(date.getMilliseconds()); //миллисекунды
```

```
alert(date.getSeconds()); //секунды
```

```
alert(date.getMinutes()); //минуты
```

```
alert(date.getHours()); //часы
```

```
alert(date.getDate()); //дни
```

```
alert(date.getMonth()); //месяцы
```

```
alert(date.getFullYear()); //год
```

Учтите, что **месяц**, который возвращает функция **getMonth**, начинается с нуля - январь нулевой, февраль первый и так далее.

Работа с getDay

С помощью объекта **Date** можно также получить номер текущего **дня недели**. Делается это с помощью метода **getDay** (он возвращает числа от 0 до 6-ти), причем

неделя начинается с **воскресенья** и этот день имеет номер **ноль**. Понедельник - это день номер **1**, вторник - номер **2** и так далее.

Как работает этот метод, посмотрим на таком примере: пусть **сегодня вторник**, тогда следующий код выведет на экран число **2**:

```
var date = new Date();
alert(date.getDay());
```

Давайте теперь выведем текущий день недели не числом, а **словом** (для краткости воскресенье будет 'вс', понедельник 'пн' и так далее).

Для этого я составлю массив дней недели **days** и начну его с воскресенья (так как это нулевой день):

```
var days = ['вс', 'пн', 'вт', 'ср', 'чт', 'пт', 'сб'];
```

Как с помощью этого массива вывести на экран, к примеру, 'вт'? Нужно **передать** в квадратных скобках номер этого дня (вторник имеет номер 2):

```
var days = ['вс', 'пн', 'вт', 'ср', 'чт', 'пт', 'сб'];
alert(days[2]); //выведет 'вт'
```

Однако, мы не хотим передавать день недели вручную, а хотим, чтобы JavaScript **сам определил** текущий день и вывел его название. Для этого нужно **совместить** то, что возвращает метод **getDay**, и наш массив **days**:

```
var date = new Date();
var day = date.getDay();
var days = ['вс', 'пн', 'вт', 'ср', 'чт', 'пт', 'сб'];
alert(days[day]);
```

Задаем определенный момент времени

Когда мы с вами создавали объект **Date()** (таким образом: **new Date()**) мы оставляли круглые скобки **пустыми**. В этом случае мы получали **текущий** момент времени.

Однако, можно поступить и по-другому - передать ему параметры в формате *new Date(год, месяц, день, часы, минуты, секунды, миллисекунды)* и в этом случае в переменную **date** запишется не текущий момент времени, а тот, который мы указали в параметрах

Особенности такого формата: отсчет месяцев начинается с нуля, отсутствующие параметры, начиная с часов считаются равными нулю, а для года, месяцев и дней — единице.

Давайте узнаем день недели (словом) за определенную дату:

```
var date = new Date(2025, 10, 5, 12, 59, 59);
var day = date.getDay();
var days = ['вс', 'пн', 'вт', 'ср', 'чт', 'пт', 'сб'];
alert(days[day]);
```

Часы, минуты и секунды можно и не указывать - это не на что не повлияет:

```
var date = new Date(2025, 10, 5);
var day = date.getDay();
```

```
var days = ['вс', 'пн', 'вт', 'ср', 'чт', 'пт', 'сб'];  
alert(days[day]);
```

Работа с getTime

При работе с датой существует специальный формат **timestamp**, который в JavaScript показывает количество миллисекунд, прошедшее с 1-го января 1970 года по текущий (или заданный) момент времени.

Существует специальный метод **getTime**, с помощью которого можно получить время в формате **timestamp**.

Получим текущий момент времени в этом формате:

```
var date = new Date();  
alert(date.getTime());
```

Получим заданный момент времени в этом формате:

```
var date = new Date(2015, 12, 4, 23, 59, 59);  
alert(date.getTime());
```

Получим разницу в миллисекундах между текущим и заданным моментом времени:

```
var now = new Date();  
var date = new Date(2015, 12, 4, 23, 59, 59);  
var diff = now.getTime() - date.getTime();  
alert(diff);
```

Работа с Date.parse

Функция **Date.parse** принимает дату в формате '*год-месяц-деньТчасы:минуты:секунды*' и возвращает соответствующей ей timestamp (количество миллисекунд, прошедших с полуночи 1 января 1970 года до заданного момента времени).

Давайте выведем количество миллисекунд, которые прошли с 1 января 1970 года до 25 мая 2015, 12:59:59:

```
document.write(Date.parse('2015-05-25T12:59:59'));
```

Результат выполнения кода:

1432547999000

Разница между датами

Даты, представленные в виде объекта Date, можно вычитать друг из друга, результат их вычитания — разница в миллисекундах.

Давайте выведем количество миллисекунд, которые прошли с 25 мая 2015, 12:59:59 по настоящий момент времени:

```
var now = new Date();  
var date = new Date(2015, 23, 4, 12, 59, 59);  
var diff = now - date;
```

```
document.write(diff);
```

Примеры решения задач

Задача

Задача. Выведите на экран текущие **день, месяц и год** в формате 'год-месяц-день'.

Решение:

```
var date = new Date();  
alert(date.getFullYear() + '-' + date.getMonth() + '-' + date.getDate());
```

Задача

Задача. Выведите на экран **текущий месяц словом**, по-русски.

Решение: создадим массив месяцев **months**, затем получим номер текущего месяца с помощью **getMonth**, и выведем месяц словом, обратившись к элементу массива **months** с ключом, равным номеру текущего месяца, вот так - **months[month]**:

```
var months = [  
    'январь', 'февраль', 'март', 'апрель', 'май', 'июнь',  
    'июль', 'август', 'сентябрь', 'октябрь', 'ноябрь', 'декабрь'  
];
```

```
var date = new Date();  
var month = date.getMonth();  
alert(months[month]);
```

Задачи для решения

Работа с new Date

*Для решения задач данного блока вам понадобятся следующие методы объекта Date: **getFullYear**, **getMonth**, **getDate**, **getHours**, **getMinutes**, **getSeconds**.*

Выведите на экран **текущий день**.

Выведите на экран **текущий месяц**.

Выведите на экран **текущий год**.

Выведите на экран текущую дату-время в формате **'12:59:59 31.12.2014'**. Для решения этой задачи **напишите функцию**, которая будет добавлять **0** перед днями и месяцами, которые состоят из одной цифры (из **1.9.2014** сделает **01.09.2014**).

Работа с getDay

Для решения задач данного блока вам понадобятся следующие методы: `getDay`.

Выведите на экран **номер текущего дня недели**.

Выведите на экран **текущий день недели** (словом, по-русски). Создайте для этого вспомогательную функцию **showDay**, которая параметром принимает число, а возвращает **день недели по-русски**.

Работа с getTime

Для решения задач данного блока вам понадобятся следующие методы: `getTime`.

Выведите на экран количество минут с **1-го января 1970 года** до **настоящего** момента времени.

Работа с Date.parse

Для решения задач данного блока вам понадобятся следующие методы: `Date.parse`.

Выведите на экран **количество часов**, прошедшее между **1 марта 1988 года** и текущим моментом с помощью **Date.parse**.

Разность между датами

Выведите на экран количество секунд с **начала дня** до **настоящего момента** времени.

Задачи

Выведите на экран количество секунд, которое осталось **до конца дня**.

Создайте **инпут**, в который пользователь вводит дату своего рождения в формате **'2014-12-31'** (с конкретным годом). По потери фокуса выведите под инпутом сколько дней осталось до его дня рождения. Воспользуйтесь методом `Date.parse`.